Bit-Exact Verification of a Parameterized INT8 Multiply–Accumulate Array for Quantized Neural-Network Inference

Rahul Nerella, Uttiyo Majumder, and Kevin Velanjickal
Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
V-SPACE FPGA Build Challenge 2026

***Abstract*—This paper describes a parameterized signed INT8 multiply–accumulate (MAC) array and a finite-state-machine-controlled quantized neural-network layer tile implemented in SystemVerilog. The design contains eight parallel output lanes, each accumulating a length-16 dot product over signed 8-bit activations and weights. A top-level controller streams a shared activation vector and lane-specific weight values into the MAC array, adds per-lane bias, applies an arithmetic right shift of seven bits, and saturates the result to signed INT8. The submitted ModelSim rerun log records three successful checks: a MAC known-answer test, a bit-exact comparison of all eight output lanes against an archived golden vector, and a second tile after re-arming the controller. The known-answer operation $3(-7) + 5(11)$ produces 34; the archived output plot reports a maximum absolute difference of zero between the recomputed Layer-0 reference and exported expected outputs. The results are simulation-level evidence for one configured Layer-0 tile, not a multi-layer inference or physical FPGA result. The archive contains Basys 3 and Nexys A7 constraint templates, but no Vivado utilization report, timing-closure report, or on-board measurements. The supplied summary graphic also labels Layers 1–4 as 100% bit-exact despite only Layer-0 test vectors being present, so those labels are treated as unverified in this paper.**

***Index Terms*—FPGA, INT8 arithmetic, multiply–accumulate, quantized neural network, fixed-point arithmetic, SystemVerilog, DSP inference, reference-model verification.**

I. Introduction

Quantized neural-network inference represents activations and weights with reduced-width integer values to reduce storage and arithmetic cost. Signed 8-bit operands are common in embedded inference pipelines because the product width and accumulator width can be bounded and tested deterministically. A hardware MAC unit must still handle sign extension, accumulation width, valid timing, bias addition, requantization, and saturation correctly. In RTL, subtle language-sizing and pipeline-alignment mistakes can pass simple compilation while producing wrong numerical results.

Experiment 3 implements a parameterized MAC unit, an array of output lanes, and a controller for one fully connected layer tile. Each activation is shared across lanes while each lane consumes its own weight for the same reduction index. The delivered verification focuses on a known arithmetic result, exact agreement with a stored output vector, and correct re-arming for a repeated tile. The report distinguishes these reproduced testbench results from synthesis assumptions and board-level behavior.

II. Design Targets and Architecture

A. Configured Workload

The packaged parameter include file `Testbench/vectors/layer0_params.svh` defines $N_{\mathrm{PAR}} = 8$, $K_{\mathrm{LEN}} = 16$, and $SHIFT = 7$. The tile therefore produces eight output values, each from sixteen signed INT8 products and one bias term. This is 128 multiply operations per tile. The default accumulator width for the testbench configuration is

$$W_{\mathrm{acc}} = 2W_{\mathrm{in}} + \lceil \log_2(ACCUM + 1) \rceil = 16 + 9 = 25 \text{ bits}, \tag{1}$$

with $W_{\mathrm{in}} = 8$ and $ACCUM = 256$. The exact parameters are module parameters rather than fixed constants in the reusable MAC modules.

B. System Datapath

Fig. 1 reproduces the architecture graphic supplied in the project ZIP. The activation and weight memories feed the lane array; the top-level controller sequences the reduction, drains the MAC pipeline, and registers the final requantized output. The term *array* refers to the eight parallel output lanes: reduction over K is sequential, with one product accumulated per active lane per cycle.

C. MAC Lane and Signed Arithmetic

The `mac_unit` computes

$$acc_g[k+1] = acc_g[k] + x[k]w_g[k], \qquad k = 0, \ldots, K_{\mathrm{LEN}} - 1, \tag{2}$$

when the enable input is asserted. The RTL explicitly sign-extends each signed input by replication before multiplication. This implementation choice was recorded in the source comments as a response to a width-cast expression that produced incorrect unsigned widening in an earlier test trace. The accumulator is reset asynchronously through `rst_ni` and synchronously cleared by `clr_i`. A two-bit valid shift register tracks the expected result-valid latency.

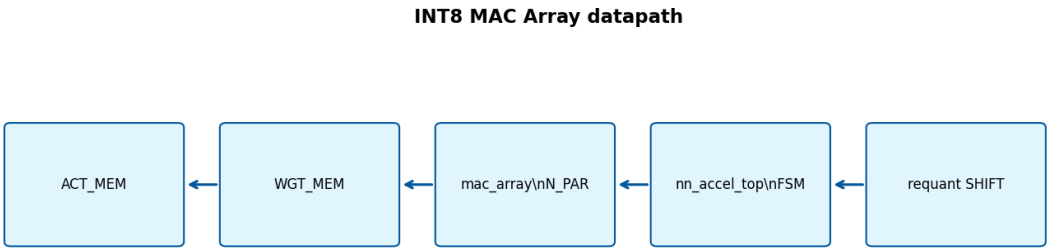


Fig. 1. Project-provided INT8 MAC-array architecture. The verified workload streams a shared activation vector and lane-specific weights through eight sequentially accumulating lanes.

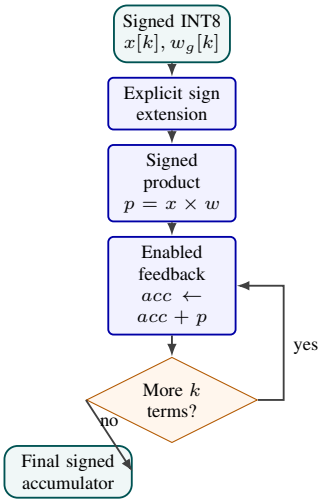


Fig. 2. Conceptual signed MAC lane. The feedback accumulator retains its value while `en_i` is low; `clr_i` clears the lane between tiles.

The source defines $OUT_WIDTH = 2W_{\text{in}} + \lceil \log_2(ACCUM + 1) \rceil$ by default and raises an elaboration-time error if the declared accumulator width is too small. This is a conservative sizing rule for the configured number of accumulations, not a substitute for proving all model-specific intermediate ranges. Although the RTL is written in a form intended to support mapping into FPGA DSP slices, no synthesis report is included to confirm the final primitive mapping or count.

D. Top-Level Tile Controller

The module `nn_accel_top` reads activation, weight, and bias arrays from memory-initialization files in simulation. The FSM sequences four states: `S_IDLE`, `S_RUN`, `S_DRAIN`, and `S_DONE`. In RUN, the index advances through K_{LEN} operands; DRAIN allows the internal valid/accumulation timing to settle; DONE exposes the result and asserts completion. The output function adds bias to the signed partial sum, performs arithmetic right shift by $SHIFT$, and saturates the value to $[-128, 127]$. No separate ReLU operation is implemented in the submitted `req` function.

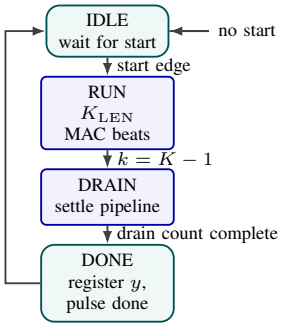


Fig. 3. Top-level controller flow represented by the four states in `nn_accel_top.sv`. One start pulse launches one tile; a subsequent pulse re-arms the controller for another tile.

III. Verification Methodology

The submitted testbench `tb_mac_array.sv` checks the standalone MAC instance and the full tile. Stimulus is applied on the falling clock edge to avoid races with positive-edge sequential logic. The full-tile test loads the included activation, weight, bias, and expected-output files using `$readmemh`. The test compares each of the eight signed INT8 outputs to its expected value exactly; it does not use a numerical tolerance.

The archive's `Simulation/rerun_2026-10-10.log` records: M1 known-answer pass; M2 full-tile bit-exact pass; M3 back-to-back tile pass; and the terminal verdict `ALL CHECKS PASSED (tb_mac_array)`. It records zero errors and zero warnings in the final simulation run, while also noting two relaxed input-port warnings at compile time in a preceding stage. ModelSim 10.5b is named in the archived compile log. This paper treats the log as captured evidence; ModelSim was not executed as part of preparing this document.

IV. Results and Discussion

A. Known-Answer and Golden-Vector Results

The standalone MAC test applies $(a, b) = (3, -7)$ followed by $(5, 11)$, producing $-21 + 55 = 34$. This result specifically exercises signed multiplication and accumulation over successive enabled cycles. The full tile produces

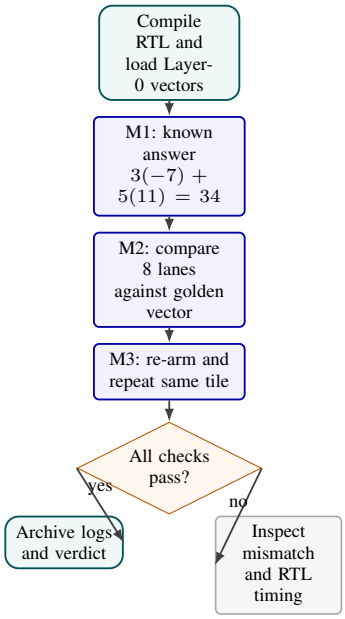


Fig. 4. Self-checking verification sequence in the submitted testbench.

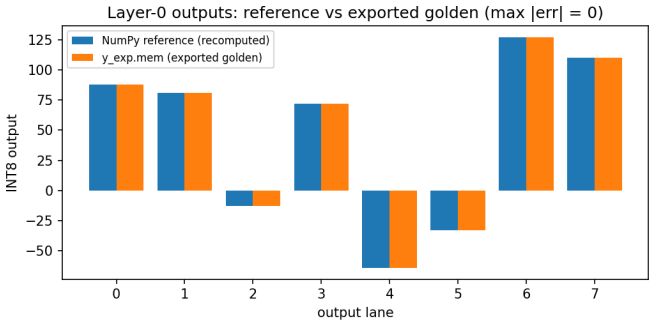


Fig. 5. Layer-0 output comparison supplied in the ZIP. The two plotted series overlap for all eight lanes; the archive labels the maximum absolute error as zero.

eight output values and compares them lane by lane against `layer0_y_exp.mem`. The result plot in Fig. 5 shows the recomputed reference and exported expected vector as equal for all displayed lanes, with the plot title reporting a maximum absolute error of zero.

B. Quantized Weight Distribution

Fig. 6 is the supplied histogram for the 128 INT8 weight values in the configured tile. The image title reports an observed range from -127 to 113. The histogram describes the distribution of the archived input weights; it does not measure accuracy or model generalization.

C. Archive Summary Plot and Evidence Scope

The ZIP also includes the project summary graphic shown in Fig. 7. Its left panel marks Layers 0 through 4 as 100% bit-exact. However, the current testbench refers only to the `layer0_*` vectors, and the archive contains no corresponding

TABLE I
EXPERIMENT 3 VERIFICATION RESULTS FROM THE ARCHIVED RERUN LOG

ID	Check	Observed result
M1	$3(-7) + 5(11)$ known-answer	34; PASS
M2	Eight outputs versus Layer-0 expected vector	Bit-exact; PASS
M3	Repeat tile after returning to IDLE	All eight outputs matched; PASS

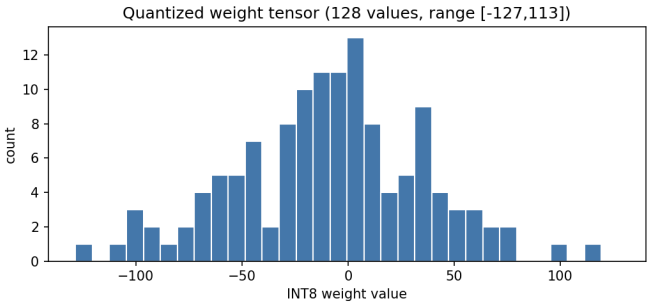


Fig. 6. Quantized weight-value histogram from the supplied Layer-0 vectors (128 entries; reported range $[-127, 113]$).

expected-output vector set for Layers 1–4. The graphic is retained to document the submitted artifact, but its layer-1 through layer-4 labels cannot be independently validated from the files in this archive. The verifiable conclusion of this paper is therefore restricted to the Layer-0 tile and the three named M1–M3 checks.

V. HARDWARE IMPLEMENTATION CONSIDERATIONS

The project includes constraint templates for Basys 3 and Nexys A7 boards, both targeting Xilinx Artix-7 families. The repository's root README correctly leaves the allocated board as TBD; the target board and pin mapping must be selected before implementation. The 100 MHz value in the project report is a target assumption, not a timing result. No Vivado synthesis/implementation log, resource utilization report, static timing report, generated bitstream, board photograph, or physical output capture is present in the supplied artifacts. Consequently, DSP48E1 count, LUT/flip-flop consumption, block-RAM inference, maximum clock frequency, and board-level functional operation remain unverified.

The current implementation is a sequential MAC array rather than a fully parallel $N \times K$ multiplier grid. For the supplied configuration, eight lanes process sixteen terms each. A future PPA study should synthesize the exact parameter set, inspect DSP and memory inference, run implementation, and report setup slack and resource counts before making throughput or area claims. Other next steps are generating vectors for additional layers, testing saturation limits and accumulator boundary cases, exercising the INT4 path mentioned

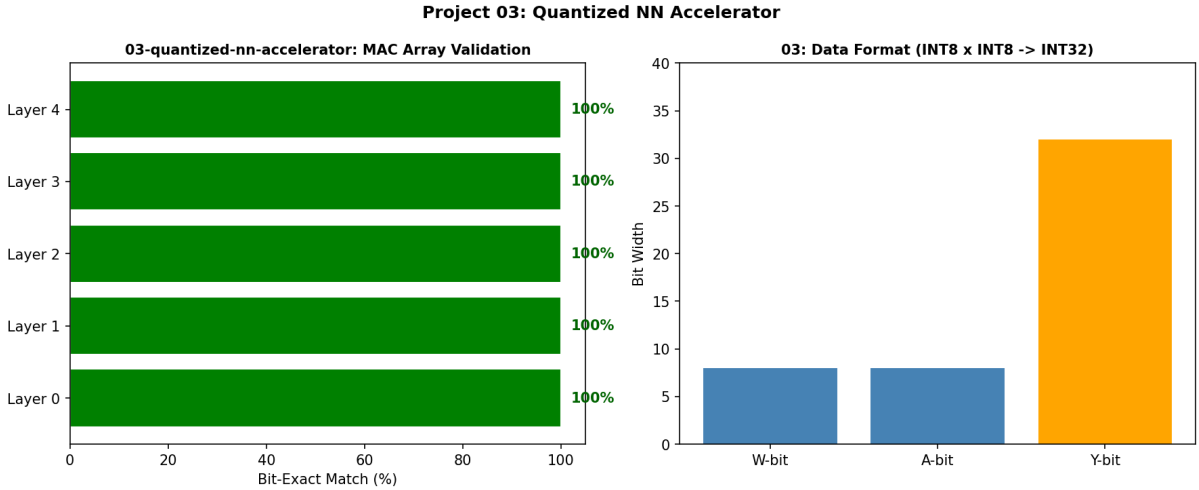


Fig. 7. Project-provided result summary. The 100% labels for Layers 1–4 are not supported by the packaged testbench vectors and are treated as unverified; the right-panel INT32 label is generic and is not the 25-bit accumulator width configured in the tested RTL. The validated numerical evidence in this paper is for Layer 0 only.

in the existing report, and exporting a waveform from a clean reproducible simulation.

VI. Applications

A parameterized INT8 MAC array is a useful building block for fully connected neural-network layers, small matrix-vector engines, and edge-inference accelerators. The same verification pattern—fixed-point arithmetic, known-answer unit tests, an independently generated golden vector, and repeated-operation tests—is applicable to DSP blocks and other accelerator datapaths. The current artifact demonstrates the arithmetic and controller behavior for one tile only; complete application-level performance would require a multi-layer scheduler, weight/activation storage policy, and target-specific implementation results.

VII. Conclusion

This paper documented a SystemVerilog INT8 MAC unit, an eight-lane sequential MAC array, and an FSM-controlled length-16 quantized layer tile. Archived ModelSim output reports successful signed known-answer arithmetic, exact agreement with the Layer-0 expected vector, and successful re-arming for a second tile. The supplied output comparison plot reports zero maximum absolute error between the recomputed and exported expected outputs. These are useful functional verification results, but they do not establish multi-layer inference or FPGA hardware operation. The summary graphic's claims for Layers 1–4 require supporting vectors and reproducible tests; synthesis, timing closure, and on-board validation also remain future work.

Acknowledgment

The authors acknowledge the V-SPACE FPGA Build Challenge 2026 for providing the structured project framework and evaluation context.

References

[1] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[2] IEEE, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[3] AMD Xilinx, *7 Series DSP48E1 Slice User Guide*, UG479. [Online]. Available: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1

[4] V-SPACE FPGA Build Challenge 2026, "Experiment 3: INT8 MAC Array," submitted RTL, testbench, vectors, simulation logs, and plots, project repository archive, Oct. 2026.